Guía del Simulink del Segway

Qué es cada bloque, qué significa cada señal y qué tocar para probar

Versión corta de `modelo_del_segway.pdf`, enfocada sólo en el diagrama

Grupo 4 — Robótica II, UNCUYO

21 de octubre de 2026

En una frase

El modelo son **seis bloques en fila**. Se lee de izquierda a derecha y lo que vuelve del último al segundo cierra el lazo. Abrís un solo archivo y apretás Run.

En un minuto

Qué abrir	`simulacion/segway_completo/INICIAR_SEGWAY.m`
Qué hacer	apretar **Run**. Construye el modelo si falta, simula y verifica
Qué tocar	la **sección 2** de ese archivo: todo lo editable está junto ahí
Dónde quedan las variables	en el **workspace base**, que es de donde Simulink las lee

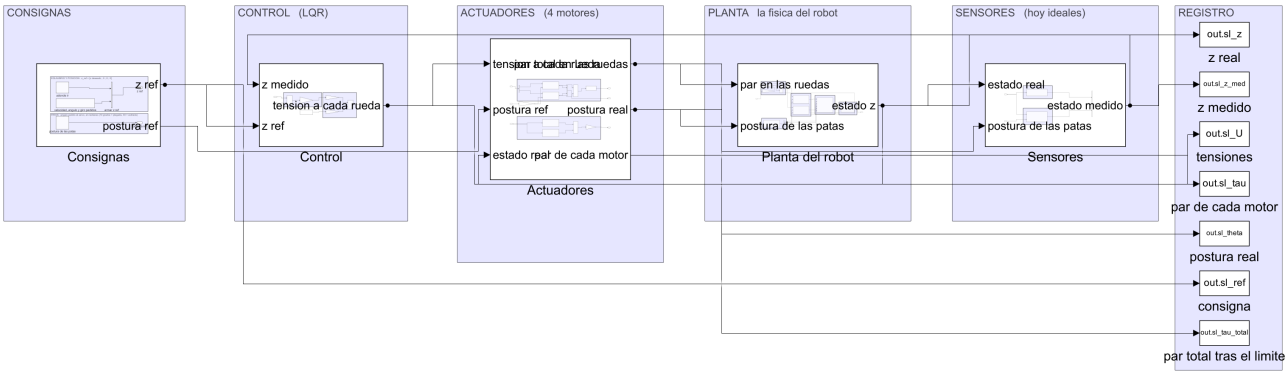
El diccionario: qué es cada símbolo

Esto es lo primero que conviene tener claro, porque el diagrama usa estos nombres en todos lados.

Símbolo	Unidad	Qué es
τ (tau)	N·m	**El par**: lo que el motor le tuerce a la rueda. Positivo **empuja hacia adelante**. Por reacción, la rueda le devuelve $-\tau$ al cuerpo, y ésa es la *única* forma que tiene el robot de inclinarse a propósito
U	V	la tensión que el control le manda a cada motor
x	m	cuánto avanzó el eje de las ruedas
v	m/s	velocidad de avance
φ (phi)	rad	**la inclinación**, medida desde la dirección del centro de masa. Positiva **hacia adelante**: el robot se inclina adelante para ir adelante
ω (omega)	rad/s	qué tan rápido se está inclinando
θ (theta)	rad	la postura de las patas: 10° plegada, 40° estirada
δ_G	rad	lo que hay *entre* el ángulo del chasis y φ. Cambia con la postura: va de −9,09° plegada a −1,78° estirada
a	kg	**masa efectiva de avance**. Vale 1,25 kg aunque el robot pese 0,92: la diferencia es la inercia del rotor vista a través de la reductora 1:20
h	kg·m	**masa por brazo**, $m_{cuerpo} \cdot \ell$. Es el *acople*: si valiera cero, avanzar e inclinarse serían independientes y el robot no se podría equilibrar con las ruedas
j	kg·m^2	**inercia del cuerpo** alrededor del eje de la rueda
k	N·m/rad	**el par de la gravedad** por radián de inclinación. Es positivo: cuanto más inclinado está, más lo voltea. Por eso el robot es inestable

El estado completo es $z = [\,x;\ v;\ \varphi;\ \omega\,]$. Ésa es la señal que viaja por casi todo el diagrama.

1. El mapa: las seis etapas

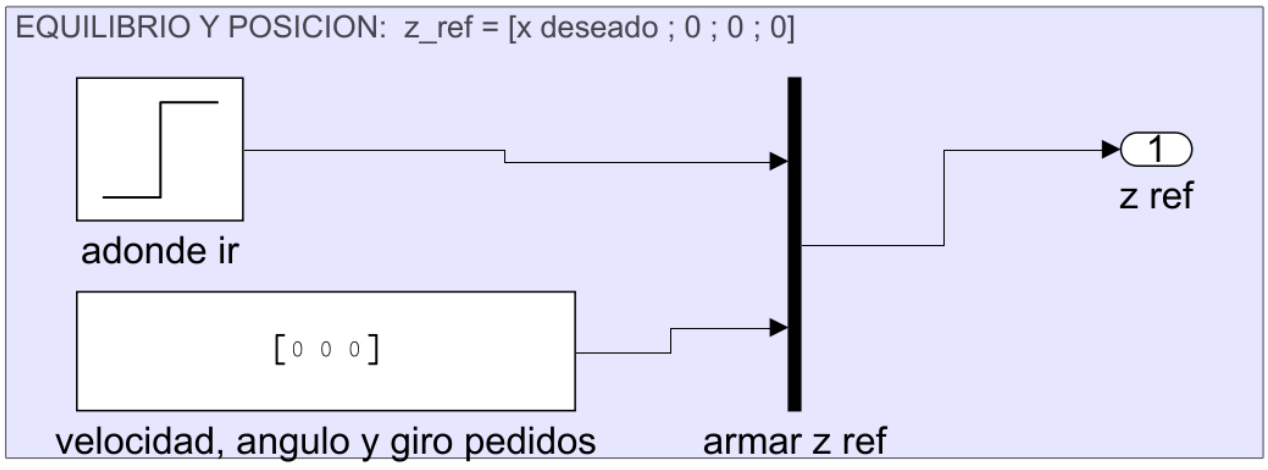


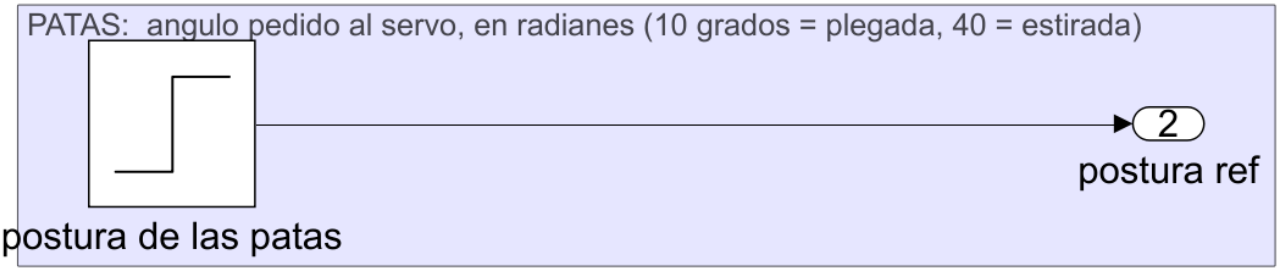
Consignas	adónde se le pide ir y en qué postura poner las patas
Control	un LQR: mira los cuatro estados a la vez y decide cuánta tensión mandarle a cada rueda
Actuadores	los cuatro motores, cada uno en su bloque
Planta	la física del robot
Sensores	lo que el robot puede medir. Hoy miden perfecto
Registro	saca las señales al workspace

Ojo

Fijate que de la planta salen **dos** caminos.
El que va a **Sensores** y de ahí al control es **lo que el robot *cree*** que le pasa. El que va directo a **Actuadores** es **lo que de verdad** le pasa, porque el motor siente la velocidad real de la rueda, no la medida.
Hoy los dos coinciden porque los sensores son ideales. Cuando les pongas ruido o retardo van a separarse, y ahí se ve si el control aguanta.

2. Consignas

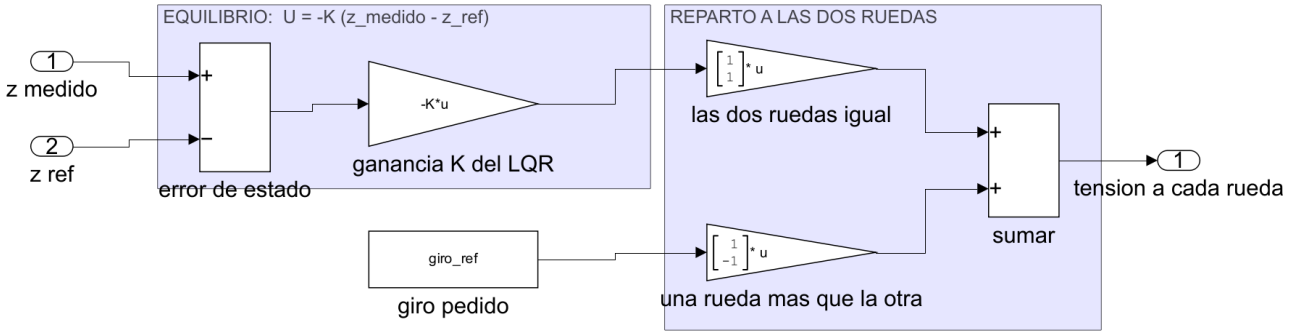




Dos escalones, nada más. Uno dice adónde ir (se arma $z_{ref} = [x_{destino}; 0; 0; 0]$: se le pide una posición y que lo demás quede en cero) y el otro en qué postura poner las patas.

Por defecto la postura es **constante**, así que el robot se comporta exactamente como el modelo matemático verificado. Poniéndole un escalón, las patas se mueven.

3. Control



En una frase

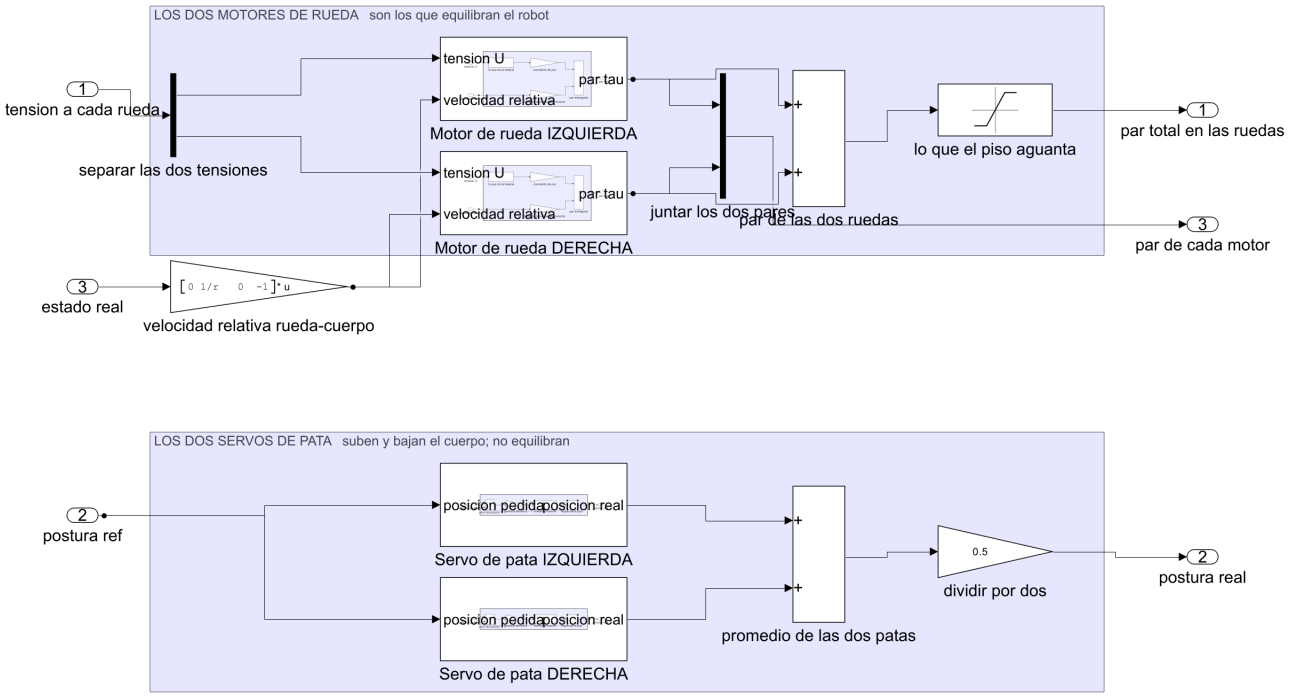
$U = -K\,(z_{medido} - z_{ref})$. Una sola multiplicación de un vector de cuatro ganancias por el error de los cuatro estados.

$K = [-55{,}5 \quad -52{,}9 \quad -90{,}7 \quad -11{,}5]$, en voltios por (m, m/s, rad, rad/s). Como $U = -K\,(z - z_{ref})$, los cuatro signos negativos quieren decir que una inclinación hacia adelante pide tensión positiva, o sea empujar hacia adelante para meterse debajo del cuerpo. Y la ganancia de posición es negativa por el cero de fase no mínima: para ir hacia adelante, primero hay que empujar hacia atrás.

Por qué una sola ganancia y no cuatro PID: el robot tiene *dos* coordenadas que importan (x y φ) y *un* solo actuador. Realimentando sólo el ángulo se equilibra pero se va caminando. K mira los cuatro estados a la vez y los pesa.

El bloque de la derecha reparte a las dos ruedas. Hoy les da lo mismo a las dos, pero la entrada de *giro* ya está puesta en cero para el día que se quiera doblar: ahí cada rueda va a recibir la suya y no hay que tocar nada más.

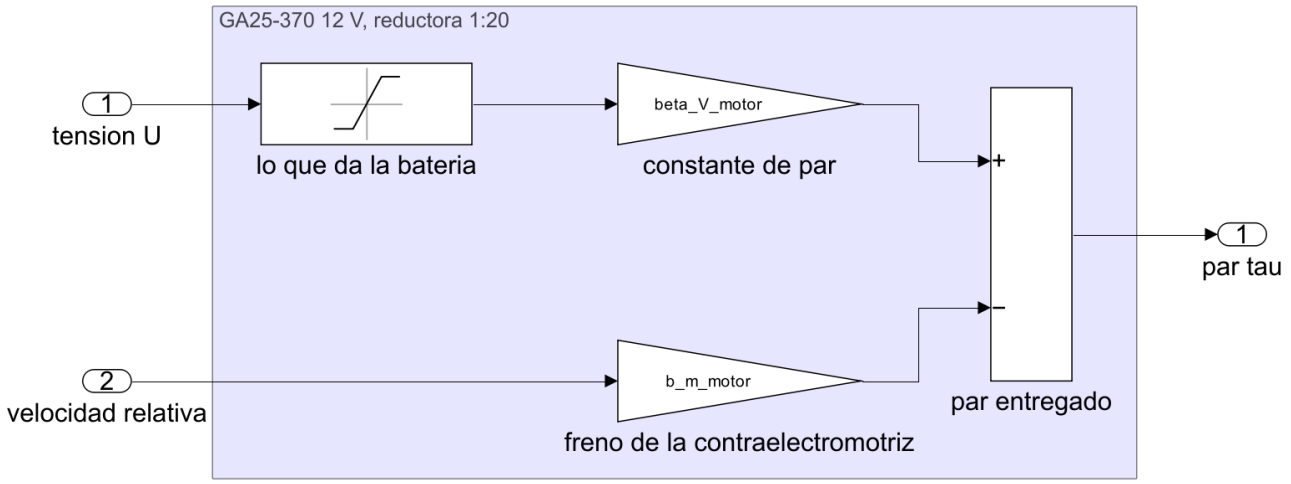
4. Actuadores: los cuatro motores



Arriba los dos motores de rueda, que son los que equilibran. Abajo los dos servos de pata, que suben y bajan el cuerpo y **no** equilibran.

El bloque de la izquierda calcula la *velocidad relativa rueda–cuerpo*, $-v/r - \omega$, que es la que ven los motores. Después de sumar los dos pares viene la saturación del *límite de agarre*: por más par que haga el motor, el piso no transmite más de lo que la fricción aguanta.

Un motor de rueda

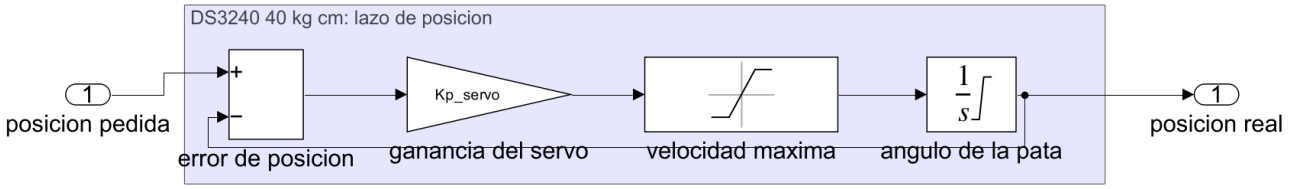


$$\tau = \underbrace{\beta_V\, U}_{\text{lo que da}} - \underbrace{b_m\, \omega_{rel}}_{\text{lo que pierde}}$$

El primer término es cuánto par da por cada voltio. El segundo es la **fuerza contraelectromotriz**: cuanto más rápido gira la rueda respecto del cuerpo, menos par entrega. Sin ese término el motor daría par infinito a cualquier velocidad; es el que le pone un techo de 1,23 m/s al robot.

La saturación de adelante es la batería: el control puede *pedir* 16 V, pero entran 11,1.

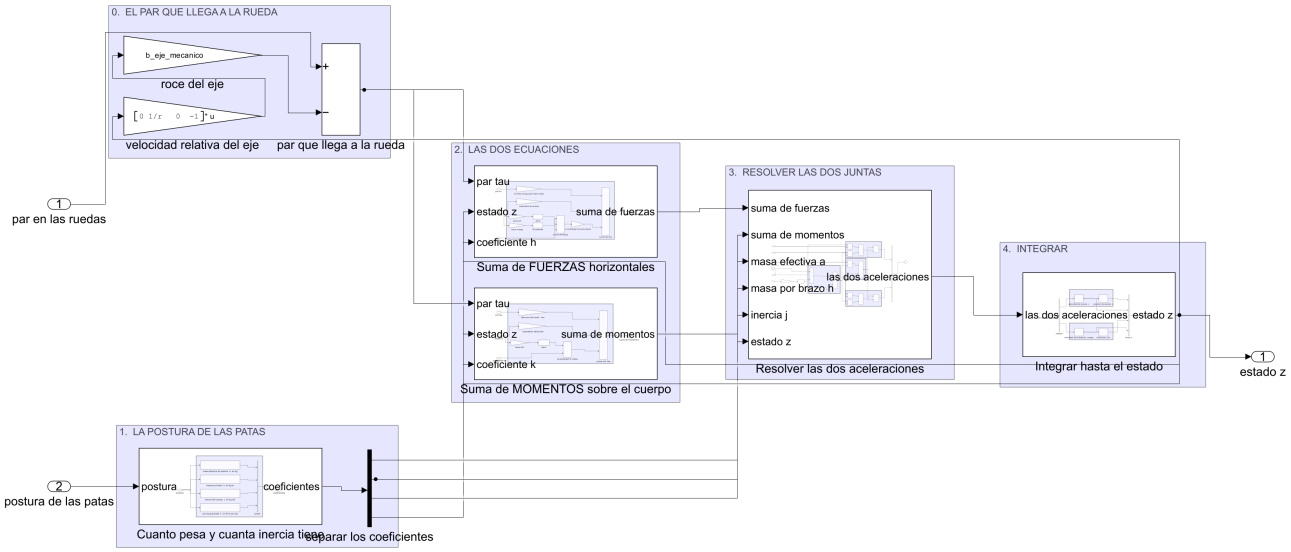
Un servo de pata



Un servo ya trae su control adentro, así que no hay que diseñarle nada: sólo decirle a qué ángulo ir. Lo que se ve es el lazo que el servo tiene de fábrica: compara la posición pedida con la que tiene y se mueve a una velocidad proporcional al error, hasta su velocidad máxima.

Los topes del integrador son los topes **mecánicos** de la pata. Que estén ahí y no en una saturación aparte evita que el ángulo se pase y después vuelva.

5. Planta: la física



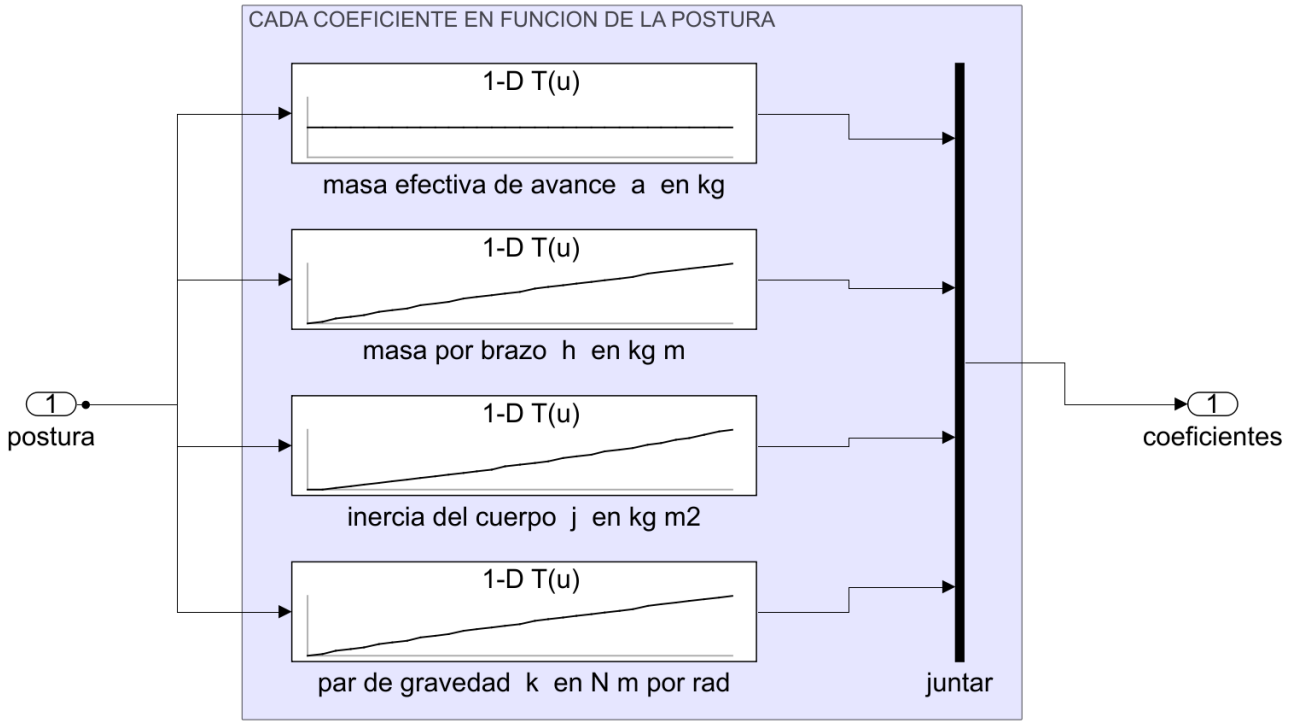
Cuatro etapas numeradas, en el orden en que se piensa el problema.

Ojo

Por qué esto no es una cadena de integradores como en clase.
En un cuerpo suelto: suma de fuerzas sobre masa da la aceleración, se integra dos veces y listo. Acá no, porque el robot son **dos cuerpos pegados**: cuando la rueda acelera hacia adelante el cuerpo se inclina hacia atrás, y cuando el cuerpo se inclina empuja la rueda.
Las dos aceleraciones aparecen en las *dos* ecuaciones. Por eso primero se arma cada suma por separado (etapa 2) y después se despejan las dos juntas (etapa 3).

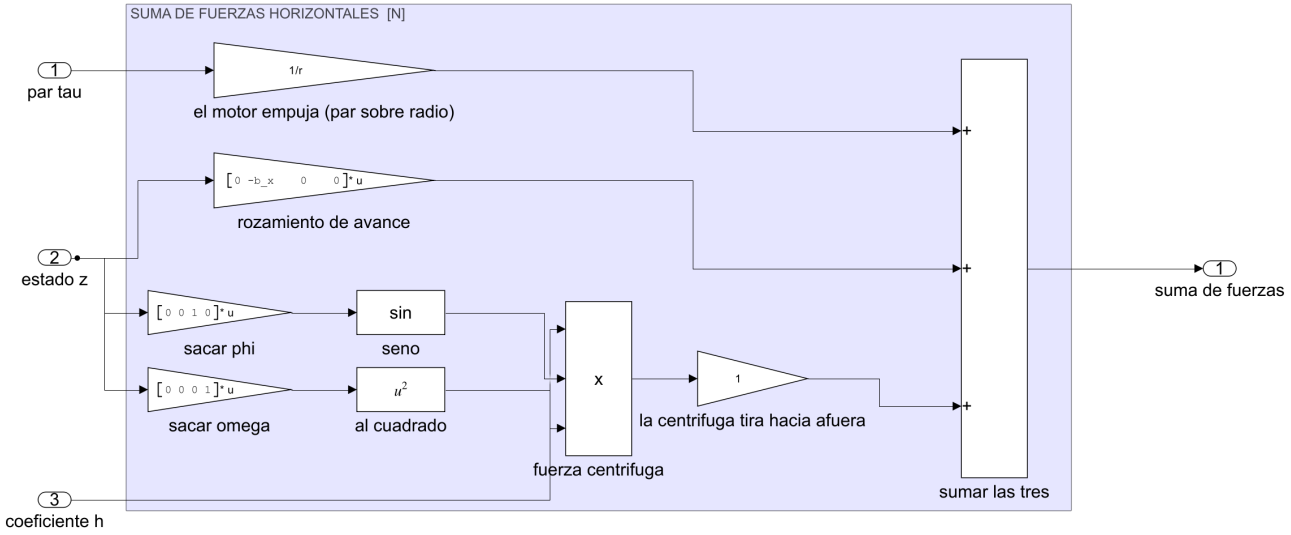
Etapa 0 y 1: el par que llega, y los coeficientes

Antes de todo se le descuenta al par el roce del eje, que es chiquito pero está.



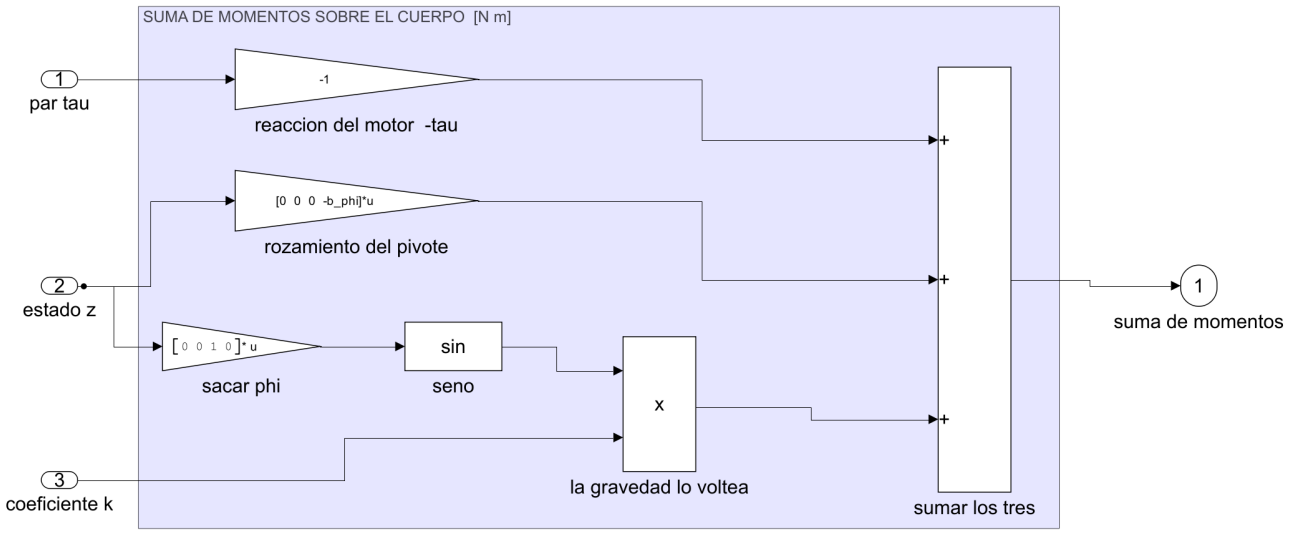
Después, la postura de las patas decide los cuatro coeficientes del cuerpo. Son tablas calculadas al arrancar desde la fuente única de parámetros: al plegarse la pata el cuerpo baja, así que cambian su brazo y su inercia. Con la postura constante devuelven siempre el mismo número.

Etapa 2: las dos ecuaciones



Suma de fuerzas horizontales, en newtons. Tres términos:

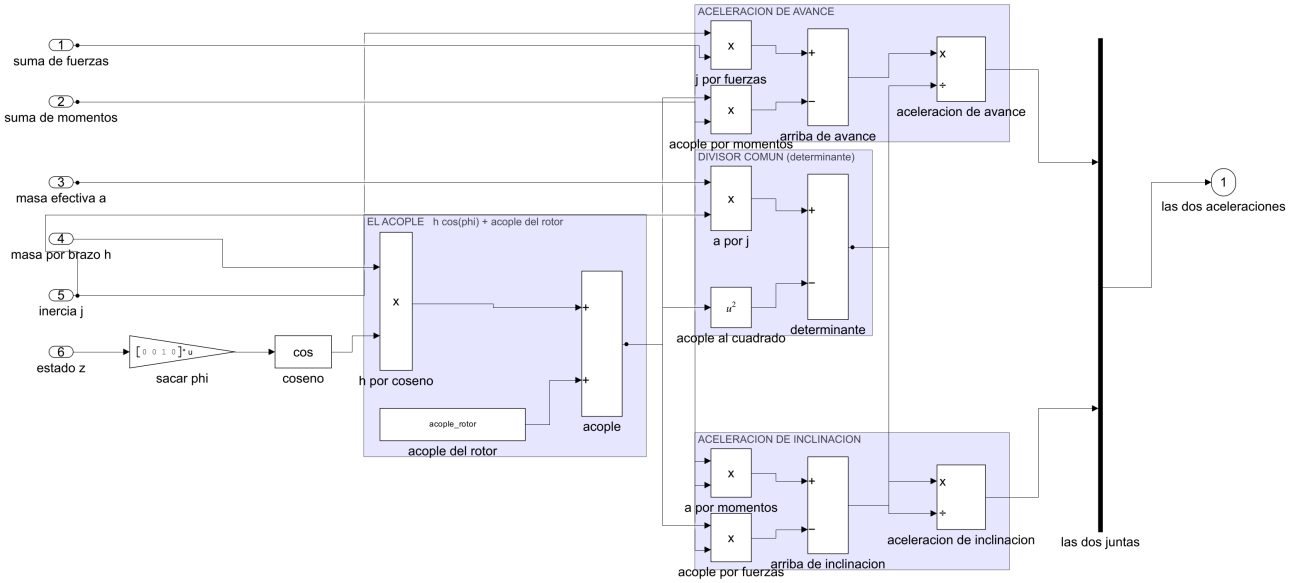
$+\tau/r$	el par del motor, visto como fuerza en el contacto con el piso
$-b_x\, v$	rozamiento de rodadura (hoy vale cero: no está medido)
$+h \sin\varphi\, \omega^2$	la fuerza centrífuga del cuerpo girando inclinado. Es el **único término no lineal** de todo el modelo, y sólo importa cuando el robot se inclina mucho y rápido



Suma de momentos sobre el cuerpo, en N·m. También tres:

$-\tau$	la **reacción** del motor. Si la rueda recibe $+\tau$, el cuerpo recibe $-\tau$. Es la única manija que tiene el control para inclinar el robot
$-b_\varphi\, \omega$	rozamiento del pivote (chiquito)
$+k \sin\varphi$	la gravedad. El signo es **positivo**: cuanto más inclinado está, más lo voltea. Ese signo es la razón de que el robot sea inestable

Etapa 3: despejar las dos aceleraciones



Hay que despejar dos incógnitas de dos ecuaciones:

$$\begin{bmatrix} a & \text{acople} \\ \text{acople} & j \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{\varphi} \end{bmatrix} = \begin{bmatrix} \text{suma de fuerzas} \\ \text{suma de momentos} \end{bmatrix}$$

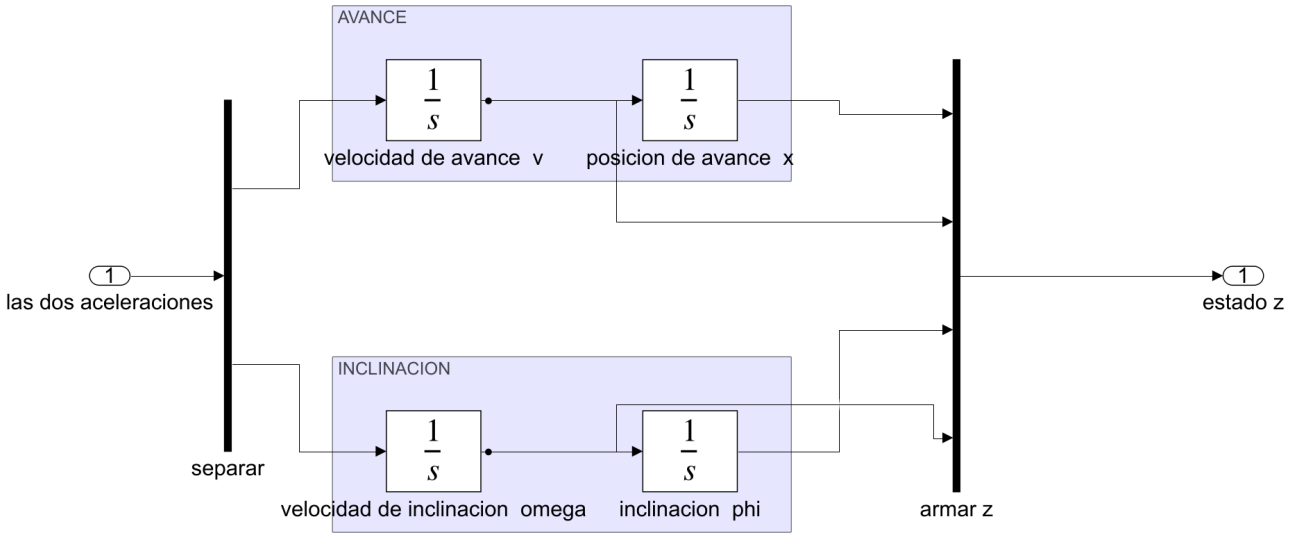
Con 2×2 se hace por **Cramer**, que es la regla de siempre: cada incógnita es un determinante dividido por el determinante común.

$$\ddot{x} = \frac{j \cdot \text{fuerzas} - \text{acople} \cdot \text{momentos}}{a\, j - \text{acople}^2}$$

$$\ddot{\varphi} = \frac{a \cdot \text{momentos} - \text{acople} \cdot \text{fuerzas}}{a\, j - \text{acople}^2}$$

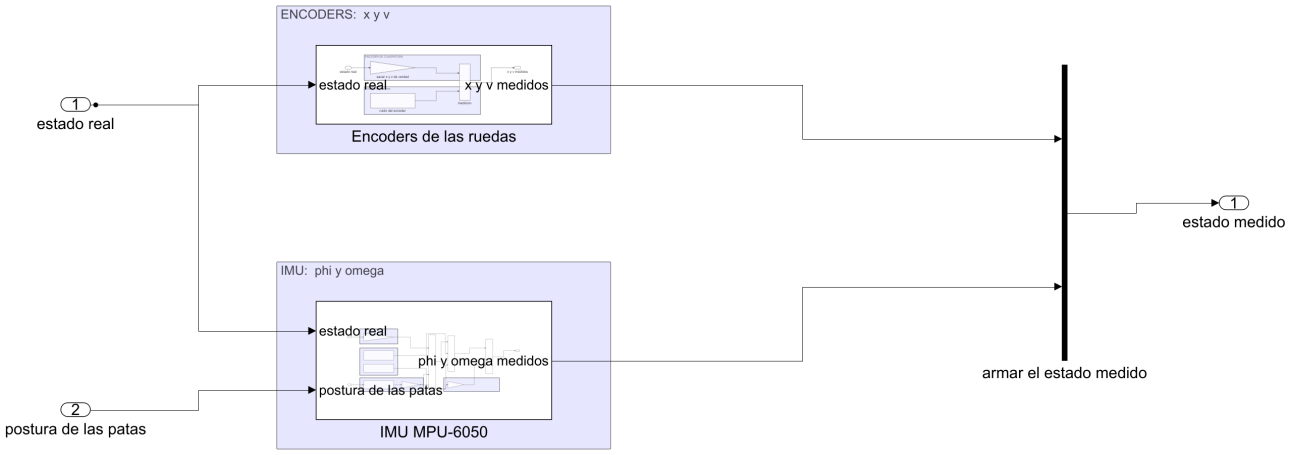
El acople vale $h \cos \varphi +$ (una constante del rotor, que es negativa), y es el número que dice cuánto se afectan entre sí avanzar e inclinarse. Si fuera cero, las dos ecuaciones se separarían y el robot no se podría equilibrar moviendo las ruedas, porque empujar la rueda no inclinaría el cuerpo.

Etapa 4: integrar



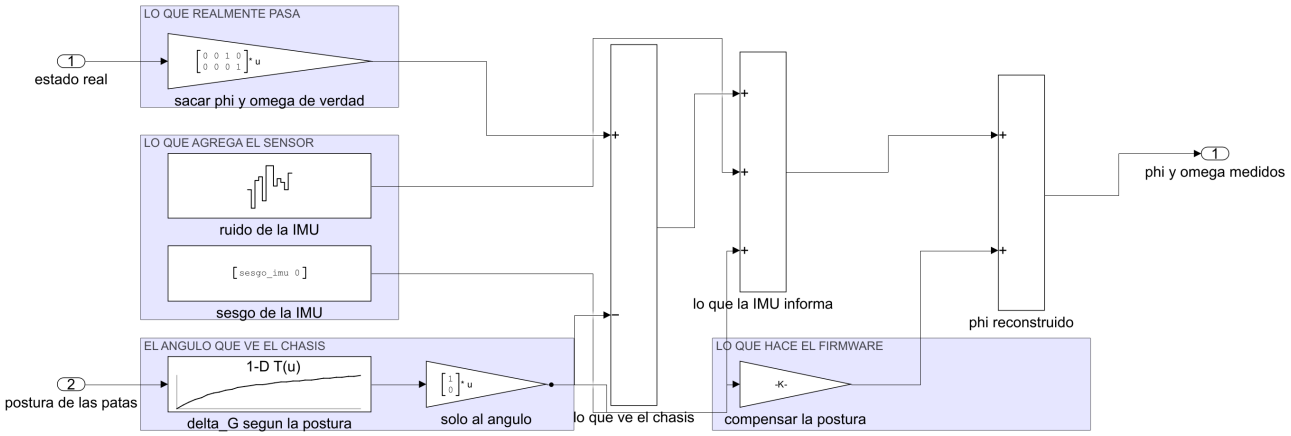
Acá sí: dos cadenas de integradores, una para el avance y otra para la inclinación. Las condiciones iniciales de los cuatro son el estado de arranque del robot.

6. Sensores



Dos sensores: los encoders de las ruedas dan x y v; la IMU da φ y ω. Hoy los dos miden perfecto y el ruido está en cero, pero los lugares donde meter la realidad ya están puestos.

La IMU, y la trampa más fácil de todo el modelo



Ojo

φ **NO es el ángulo del chasis.**
φ se mide desde la dirección del centro de masa, y entre el chasis y esa dirección hay un ángulo δ_G. El problema es que δ_G **cambia con la postura de las patas**: vale 9,09° con la pata plegada y 1,78° con la pata estirada.
O sea que al mover las patas, **el cero del ángulo se corre más de 7 grados**.

Por eso el bloque tiene tres partes: lo que realmente pasa, lo que le agrega el sensor (ruido y sesgo), y lo que hace el firmware, que es volver a sumarle δ_G para recuperar φ.

Si el firmware no lo compensa, el control cree que el robot está inclinado cuando no lo está y sale a corregir: el robot se va **21 cm** de donde estaba cada vez que cambia de postura. Compensando, se queda en cero.

7. Cómo correrlo

Abrís `INICIAR_SEGWAY.m` y apretás Run. Lo que conviene tocar está todo junto en la **sección 2**:

Variable	Por defecto	Qué cambia
`postura_deg`	25	en qué postura se diseña el control
`phi0_deg`	5	de qué inclinación arranca
`x_destino`	0,30 m	adónde se le pide ir (0 = quedarse quieto)
`theta_pata_fin`	igual a la inicial	si las patas se mueven o no
`giro_ref`	0	diferencia de tensión entre las dos ruedas
`ruido_imu_angulo`	0	ruido del ángulo de la IMU
`ruido_encoder_x`	0	ruido de posición del encoder
`sesgo_imu`	0	error constante de la IMU, en radianes
`compensar_delta_G`	1	si el firmware corrige el ángulo según la postura

Al terminar, todo queda en el workspace base: `p` (la planta), `motor`, `C` (el diseño del control con sus polos y márgenes), `K`, y los resultados `z_sl`, `z_med`, `U_sl`, `tau_sl`, `th_sl`.

Por qué se puede confiar en el resultado

Simulink no se valida solo. El mismo problema se integra en paralelo con `ode45` sobre una implementación *independiente* de la planta, escrita a mano. Las dos coinciden en $1{,}3 \times 10^{-12}$ m de posición y $8{,}3 \times 10^{-12}$ rad de ángulo. Además hay **11 tests** propios, y la suite del proyecto está en **35 de 35**.

Para probar

Cuatro experimentos que se hacen cambiando un número:

1. **Mover las patas.** Poné `theta_pata_fin = 40` con `postura_deg = 10`. El robot se estira mientras se equilibra.
2. **Ver la trampa de δ_G.** Lo mismo pero con `compensar_delta_G = 0`. Se va 21 cm.
3. **Poner ruido.** `ruido_imu_angulo = 1e-6`. Ahí empiezan a separarse lo que el robot cree y lo que pasa.
4. **Hacerlo doblar.** `giro_ref = 1`. Hoy el modelo no representa el giro, así que vas a ver que las dos ruedas reciben distinto pero la planta no se entera: eso es lo que falta agregar.

Lo que todavía NO está en el modelo

Para que quede claro hasta dónde llega:

período de muestreo	el modelo es continuo. El control real va a correr cada 5 ms
retardo de cálculo	un paso entre medir y actuar
cuantización del encoder	0,24 mm por cuenta
giro	las dos ruedas pueden recibir distinto, pero la planta es plana sagital
patas con inercia	las tablas congelan la planta en cada postura e interpolan

Los tres primeros importan: con ellos, el LQR de libro quedaba **inestable** y hubo que repesarlo. Está contado en el capítulo 7 de `modelo_del_segway.pdf`.